

Relatório Técnico: Análise e Melhoria do Sistema Proj_tocai

Autor: Manus AI **Data:** 26 de Outubro de 2025 **Revisão:** 1.0

1. Introdução

Este relatório detalha o processo de análise, correção de erros e implementação de melhorias nas funcionalidades centrais do sistema **Proj_tocai**, conforme solicitado pelo usuário. O foco principal foi garantir o correto funcionamento das funcionalidades de **Avaliação**, **Denúncias**, **Favoritos** e **Notificação de Match**, além de sugerir e implementar uma nova funcionalidade de valor.

2. Análise da Estrutura do Projeto

O projeto é dividido em dois componentes principais:

- **Backend:** Desenvolvido em **TypeScript** com **Node.js** e **Express**, utilizando **TypeORM** para a camada de acesso a dados.
- **Frontend:** Desenvolvido em **Vue.js** com **TypeScript**, utilizando **Pinia** para gerenciamento de estado.

3. Erros Encontrados e Soluções Implementadas (Fase de Correção)

A fase inicial de compilação e teste revelou diversos erros, principalmente de tipagem e importação, que impediam o funcionamento adequado do sistema.

Componente	Erro Encontrado	Solução Implementada
Backend	Erros de importação e tipagem em middlewares (<code>role.middleware.ts</code>) e rotas (<code>report.routes.ts</code> , <code>userRoutes.ts</code>).	Correção das importações, trocando referências incorretas como <code>authorizeRole</code> por <code>roleMiddleware</code> .
Backend	Falta da classe <code>ForbiddenError</code> em <code>src/errors/http-errors.ts</code> .	Adição da definição da classe <code>ForbiddenError</code> em <code>src/errors/http-errors.ts</code> .
Backend	Erro de referência em <code>src/middlewares/error.handler.ts</code> (<code>error.status</code>).	Correção da referência para <code>error.statusCode</code> para seguir o padrão das classes <code>HttpError</code> .
Backend	Erro de enum em <code>src/entities/Report.ts</code> (<code>ReportStatus.pending</code>).	Correção para o valor correto do enum, <code>ReportStatus.PENDENTE</code> .
Frontend	Arquivos de entidade do TypeORM (<code>Item.ts</code> , <code>User.ts</code> , etc.) localizados incorretamente no diretório <code>src/components</code> .	Remoção dos arquivos de backend do diretório do frontend.
Frontend	Erros de tipagem em componentes e stores devido à falta de exportação de tipos (<code>User</code> , <code>AuthResponse</code> , <code>Rating</code> , etc.) no arquivo <code>src/types/index.ts</code> .	Correção das importações para usar <code>from '@types/index'</code> e garantia de que todos os tipos necessários estão exportados.
Frontend	Erros de referência a propriedades de <i>loading</i> nas stores (<code>itemStore.loading</code>).	Correção das referências para as propriedades corretas (<code>itemStore.loadingFetchItems</code> , <code>itemStore.loadingFetchMyItems</code>).
Frontend	Erros de tipagem em <code>src/views/ItemDetailsView.vue</code> e <code>src/views/AdminView.vue</code> (<code>item.owner</code> e <code>user.createdAt</code> possivelmente <code>undefined</code>).	Adição de operadores de encadeamento opcional (<code>?.</code>) e operadores de não-nulo (<code>!</code>) para garantir a segurança da tipagem.
Frontend	Componente <code>PreferredItemCard.vue</code> usado incorretamente para exibir apenas o título da preferência.	Criação do novo componente <code>PreferredTitleCard.vue</code> para exibir a preferência de forma mais simples e correta.

4. Melhorias Implementadas nas Funcionalidades Chave

As seguintes melhorias foram aplicadas para aumentar a robustez e a usabilidade das funcionalidades solicitadas:

4.1. Favoritos (Backend)

Melhoria	Detalhes
Carregamento de Relações	A função <code>favoriteService.findByUser</code> foi aprimorada para carregar as relações <code>item.tradePreferences</code> e <code>item.imagens</code> , garantindo que o frontend receba dados completos para exibição.
Consistência de Dados	O <code>favoriteController.listMyFavorites</code> foi ajustado para filtrar itens nulos, prevenindo erros caso um item favorito tenha sido deletado.

4.2. Avaliação (Backend)

Melhoria	Detalhes
Prevenção de Duplicidade	A função <code>ratingService.create</code> agora verifica se já existe uma avaliação para a mesma proposta (<code>proposalId</code>) feita pelo mesmo usuário (<code>fromUserId</code>), prevenindo avaliações duplicadas.

4.3. Denúncias (Backend)

Melhoria	Detalhes
Prevenção de Denúncias Pendentes Duplicadas	A função <code>reportService.createReport</code> foi aprimorada para verificar se já existe uma denúncia com status PENDENTE para o mesmo alvo (usuário ou item) feita pelo mesmo denunciante. Isso evita a sobrecarga do sistema com denúncias repetidas.

4.4. Notificações (Backend)

Melhoria	Detalhes
Link Direto no Match	A notificação de <code>MATCH_FOUND</code> agora inclui um link direto para o item desejado (<code>/items/:itemId</code>), facilitando a ação imediata do usuário.
Metadata no Match	Adição do <code>itemId</code> no metadata da notificação de match, permitindo que o frontend exiba informações mais ricas.

5. Sugestão e Implementação de Nova Funcionalidade: Match Bidirecional Aprimorado

Sugestão: Aprimorar o sistema de notificação de match para identificar e notificar sobre **Matches Perfeitos (Bidirecionais)**.

Implementação:

A função `itemService.findMatchesAndNotify` foi modificada para incluir a lógica de match bidirecional:

- Match Unidirecional:** Um item existente (B) deseja o título do item recém-criado (A). O dono de B é notificado.
- Match Bidirecional (Perfeito):** Além do match unidirecional, o item recém-criado (A) também deve ter o título do item existente (B) listado em suas preferências de troca.
- Notificação Aprimorada:**
 - Se for um match unidirecional, a notificação padrão é enviada.

- Se for um **Match Perfeito**, ambos os donos (A e B) recebem uma notificação com um título especial (✨ Match Perfeito Encontrado! ✨) e uma mensagem mais enfática, incentivando a troca imediata.

Esta melhoria aumenta significativamente a chance de conversão de trocas ao destacar as oportunidades mais promissoras para o usuário.

6. Conclusão

O sistema **Proj_tocai** foi corrigido e aprimorado, garantindo que as funcionalidades de avaliação, denúncias, favoritos e notificação de match estejam operacionais e mais robustas. A implementação do **Match Bidirecional** adiciona um valor significativo à experiência do usuário, tornando o sistema de trocas mais inteligente e eficiente.

O projeto final, com todas as correções e melhorias, está pronto para ser empacotado e entregue.